

# DeviceCare

## Hospital Computerized Maintenance Management System



**Document Type:**  
Project Documentation

**Stage:**  
Preliminary Defense

**Supervised by:** Eng. Ezz El-Din

**Prepared by:** Sondos Mohammed

## 1. Abstract

DeviceCare is a full-stack Hospital Computerized Maintenance Management System (CMMS) designed to digitize and automate the complete lifecycle of medical device management within healthcare facilities. The system addresses a critical operational gap faced by biomedical engineering departments: the reliance on paper-based records and manual processes for tracking device contracts, maintenance histories, and fault reports.

By consolidating device information, maintenance workflows, contractual obligations, and interdepartmental communication into a single digital platform, DeviceCare significantly reduces device downtime, ensures regulatory compliance, and establishes a clear accountability chain across all stakeholders. The system generates QR codes for each device, enabling instant access to complete device records on-site, and delivers automated scheduling notifications to ensure that preventive maintenance and calibration deadlines are never missed.

Built on a modern MERN-adjacent stack (React.js, Node.js/Express.js, MongoDB), DeviceCare supports four distinct user roles — Administrator, Biomedical Engineer, Technician, and Staff — each with tailored access permissions and workflow responsibilities, forming a structured three-tier maintenance pipeline from fault reporting through to resolution documentation.

## 2. Introduction

Medical devices are the backbone of patient care in modern hospitals. Their operational reliability directly impacts clinical outcomes and patient safety. Hospitals typically manage hundreds of devices across numerous departments, each requiring regular preventive maintenance, calibration, contractual oversight, and fault response. In Egyptian public and private hospitals, this management is predominantly handled through paper-based job sheets, physical filing systems, and informal verbal communication — a model that is inherently error-prone, difficult to audit, and incompatible with the demands of a modern healthcare environment.

This project originated from direct professional experience in a hospital biomedical engineering department, where recurring operational challenges highlighted the urgent need for a digital management solution. DeviceCare was conceived and developed to bridge this gap — providing a centralized, role-aware, and fully auditable system that transforms how hospitals manage their medical equipment infrastructure.

**The project encompasses the following core scope:**

- Full-stack web application accessible via any browser
- Backend REST API with 40+ secured endpoints
- Role-based access for four stakeholder types
- Automated maintenance scheduling and notification system
- QR code-based physical device identification
- Real-time communication channel between staff and engineering team
- Complete audit trail for all device interactions

## 3. Problem Definition

The following problems were identified through direct field experience in a hospital biomedical engineering department and form the foundation for the DeviceCare system:

### 3.1 Contract Visibility and Device Warranty Status

When a biomedical engineer or technician was assigned to inspect or repair a device, there was no quick way to determine whether that device was covered under a maintenance contract. Devices under active contracts must not be opened or serviced by internal staff, as this voids the contract terms. Determining this required manually searching through physical contract binders and cross-referencing device serial numbers — a process that could take hours and was prone to error.

#### DeviceCare Solution

Each device record in the system stores a direct reference to its associated maintenance contract (company name, start date, end date, contact phone). This information is instantly visible on the device profile and via QR code scan, eliminating the need for paper searches entirely.

### 3.2 QR Code-Based Device Identification

Staff members needed a fast, reliable method to access complete device information on the floor without going to an office or calling the engineering department. Previously, this required phone calls or physical searches through records.

#### DeviceCare Solution

The system automatically generates a unique QR code for every registered device. This QR code is printed and affixed directly to the device. Scanning it instantly displays the complete device profile: name, serial number, department, status, contract details, last maintenance date, and maintenance history.

### 3.3 Job Sheet and Maintenance Record Loss

After a biomedical engineer completes a service visit, they issue a paper job sheet as proof of work performed. These job sheets frequently get lost or misplaced, creating serious issues during hospital inspections when inspectors request the most recent job sheet for a specific device. Staff would spend significant time searching through physical files, often unsuccessfully.

#### DeviceCare Solution

Engineers can upload a scanned image of the job sheet directly into the device's record in the system upon completion. All uploaded documents are stored in Cloudinary and are retrievable through the device's media gallery, filterable by date. The most recent job sheet is always accessible within seconds.

### 3.4 Untracked Fault Reporting and Accountability

When a device malfunctioned, nursing or medical staff would report the issue verbally or by phone. There was no formal documentation of who reported the fault, when it was reported, what the specific issue was, or who ultimately serviced the device. This lack of traceability frequently led to disputes over accountability and made it impossible to identify devices with recurring fault patterns.

#### **DeviceCare Solution**

Staff submit structured fault reports through the system's maintenance request module. Each request captures: the reporter's identity, timestamp, issue description, priority level, and the specific device. The request is immediately visible to the Biomedical Engineer and Administrator, who then create a work order assigning a technician. All actions are logged, creating a complete and immutable audit trail.

### **3.5 Missed Preventive Maintenance and Calibration Schedules**

Preventive maintenance (PM) and calibration schedules for medical devices are legally and clinically mandated. However, without a centralized scheduling system, these deadlines were frequently missed due to workload, oversight, or staff turnover. Missed PMs expose hospitals to regulatory penalties and increase the risk of device failure during patient care.

#### **DeviceCare Solution**

The system uses an automated cron job (node-schedule) that runs daily and sends notifications to the Administrator, Biomedical Engineer, and relevant Technician at the start of each day, listing all upcoming or overdue calibrations and preventive maintenance tasks. This ensures zero missed schedules and creates a documented record of all PM activities.

### **3.6 No Mechanism for Identifying End-of-Life Devices**

Hospitals are required to conduct annual device inventories and identify equipment that has exceeded its operational lifespan (typically 10+ years) for decommissioning. Without a digital system, generating this report required manual cross-referencing of purchase dates across hundreds of paper records.

#### **DeviceCare Solution**

The system stores the purchase date for every device and provides filterable reports. An exportable Excel sheet can be generated listing all devices by age, flagging those exceeding 10 years for decommissioning review. This reduces the annual inventory process from days to minutes.

## **4. System Architecture & Technologies**

### **4.1 High-Level Architecture**

DeviceCare follows a standard three-tier web application architecture:

|                          |                                                                                                                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Presentation Tier</b> | React.js (Vite) with Tailwind CSS — renders the UI, handles routing via React Router DOM, and communicates with the backend via Axios HTTP client.                                          |
| <b>Application Tier</b>  | Node.js with Express.js — processes all business logic, enforces role-based access control, manages authentication, and orchestrates external services (Cloudinary, Nodemailer, Socket.io). |
| <b>Data Tier</b>         | MongoDB — document-oriented NoSQL database managed via Mongoose ODM, storing all entities including users, devices, departments, contracts, maintenance records, and work orders.           |

## 4.2 Technology Stack

| Layer    | Technology           | Version | Purpose             |
|----------|----------------------|---------|---------------------|
| Frontend | React.js (Vite)      | ^18.x   | UI Framework        |
| Frontend | Tailwind CSS         | ^3.x    | Styling             |
| Frontend | React Router DOM     | ^6.x    | Client-side Routing |
| Frontend | Axios                | ^1.x    | HTTP Client         |
| Backend  | Node.js + Express.js | ^5.x    | REST API Server     |
| Backend  | MongoDB + Mongoose   | ^9.x    | Database + ODM      |
| Backend  | JSON Web Tokens      | ^9.x    | Authentication      |
| Backend  | bcrypt               | ^6.x    | Password Hashing    |
| Backend  | Joi                  | ^18.x   | Input Validation    |
| Backend  | Nodemailer           | ^8.x    | Email Service       |
| Backend  | Socket.io            | ^4.x    | Real-time Chat      |
| Backend  | node-schedule        | ^2.x    | Cron Jobs           |
| Cloud    | Cloudinary           | ^2.x    | Image Storage       |
| Cloud    | QRCode (npm)         | ^1.x    | QR Generation       |

## 4.3 Role-Based Access Control

The system enforces four distinct user roles with differentiated permissions:

|                            |                                                                                                                              |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Administrator</b>       | Full system access: manage users, devices, departments, contracts, and view all maintenance records.                         |
| <b>Biomedical Engineer</b> | Reviews maintenance requests, creates and assigns work orders, monitors device health, and manages the maintenance schedule. |
| <b>Technician</b>          | Executes assigned work orders, logs maintenance records, updates device status, and uploads job sheet documentation.         |

**Staff  
(Nurse/Doctor)**

Submits maintenance requests for faulty devices, accesses device information via QR scan, and communicates with the engineering team via the in-app chat.

## 4.4 Maintenance Workflow (3-Tier Pipeline)

The core operational flow of the system follows a structured three-tier pipeline:

|   |                                        |                                                                                                                                                                                                     |
|---|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Fault Report (Staff)</b>            | Staff member submits a MaintenanceRequest with issue description, priority (Normal/Emergency), and device ID. System captures reporter identity and timestamp.                                      |
| 2 | <b>Work Order (Biomed Engineer)</b>    | Biomedical Engineer reviews the request and creates a WorkOrder, assigning it to a specific Technician. Request status automatically updates to “In Progress.”                                      |
| 3 | <b>Maintenance Record (Technician)</b> | Technician executes the work order, logs a Maintenance record (type, notes, status), and upon completion the device status automatically updates to “Working” and the request is marked “Resolved.” |

## 5. Implementation & Dependencies

### 5.1 Backend API

The backend exposes a RESTful API organized into the following modules:

| Module      | Endpoint                      | Method | Role       | Description        |
|-------------|-------------------------------|--------|------------|--------------------|
| Auth        | POST /api/register            | POST   | Public     | Register new user  |
| Auth        | POST /api/login               | POST   | Public     | Login + JWT        |
| Auth        | GET /api/confirmEmail/:token  | GET    | Public     | Confirm email      |
| Auth        | POST /api/forget              | POST   | Public     | Forgot password    |
| Auth        | POST /api/reset/:token        | POST   | Public     | Reset password     |
| Auth        | GET /api/profile              | GET    | All Auth   | Get user profile   |
| Devices     | POST /api/device              | POST   | Admin      | Add device         |
| Devices     | GET /api/device               | GET    | All Auth   | List all devices   |
| Devices     | PATCH /api/device/:id         | PATCH  | Admin      | Update device      |
| Devices     | DELETE /api/device/:id        | DELETE | Admin      | Delete device      |
| Maint. Req. | POST /api/maintenance-request | POST   | Staff      | Submit request     |
| Maint. Req. | GET /api/maintenance-request  | GET    | Eng/Admin  | View all requests  |
| Work Order  | POST /api/work-order          | POST   | BiomedEng  | Create work order  |
| Maintenance | POST /api/maintenance         | POST   | Technician | Log maintenance    |
| Departments | GET/POST /api/department      | CRUD   | Admin/Auth | Manage departments |
| Contracts   | GET/POST /api/contract        | CRUD   | Admin/Auth | Manage contracts   |



## 5.2 Database Schema

The system uses 7 MongoDB collections with the following key relationships:

- User → stores identity, role, hashed password, email confirmation status, and Cloudinary profile image
- Device → references Department and Contract via ObjectId; stores status, priority, and purchase date
- Department → stores name and physical location
- Contract → stores company name, contract period (startDate/endDate), and phone number
- MaintenanceRequest → references Device (ObjectId) and createdBy (User ObjectId)
- WorkOrder → references MaintenanceRequest, assignedBy (BiomedEng), and assignedTo (Technician)
- Maintenance → references Device and WorkOrder; records type, date, status, and notes

## 5.3 Authentication & Security

- JWT tokens with role payload, 1-hour expiry for session tokens
- Email confirmation via Nodemailer: 5-minute token for initial confirmation, 1-day token for resend
- Password hashing via bcrypt with salt rounds 8–10
- Forgot/Reset password: nanoid-generated code, hashed and stored as forgetCode, delivered via email link
- Auth middleware implemented as a Higher-Order Function: auth() for any authenticated user, auth([roles]) for role-specific routes
- Joi validation middleware on all request bodies and route parameters

## 5.4 External Dependencies & Third-Party Integrations

| Dependency     | Purpose                                                               | Version |
|----------------|-----------------------------------------------------------------------|---------|
| Cloudinary SDK | Cloud-based image storage for profile pictures and job sheet uploads  | ^2.10.0 |
| Nodemailer     | SMTP email delivery for account confirmation and password reset links | ^8.0.7  |
| Socket.io      | WebSocket-based real-time bidirectional communication for in-app chat | ^4.8.3  |
| node-schedule  | Cron-based job scheduler for automated daily maintenance reminders    | ^2.1.1  |
| QRCode (npm)   | Server-side generation of QR codes encoding device profile data       | ^1.5.4  |
| jsonwebtoken   | JWT creation and verification for authentication tokens               | ^9.0.3  |

|        |                                                                        |         |
|--------|------------------------------------------------------------------------|---------|
| bcrypt | Secure password hashing with configurable salt rounds                  | ^6.0.0  |
| Joi    | Schema-based request validation for all API endpoints                  | ^18.2.1 |
| Multer | Multipart form-data parsing for file upload handling                   | ^2.1.1  |
| nanoid | Cryptographically secure unique ID generation for password reset codes | ^5.1.16 |
| dotenv | Environment variable management for configuration secrets              | ^17.4.2 |

## 6. Achievements & Progress

The following table summarizes the implementation status of all planned features as of the preliminary defense stage:

| Feature / Module                        | Status        | Notes                                |
|-----------------------------------------|---------------|--------------------------------------|
| Complete Backend API (40+ endpoints)    | ✓ Done        | All modules tested in Postman        |
| JWT Authentication + Email Confirmation | ✓ Done        | 5-min + 1-day resend tokens          |
| Forgot / Reset Password Flow            | ✓ Done        | Secure token via email link          |
| Role-Based Access Control (4 roles)     | ✓ Done        | Middleware HOF pattern               |
| Device Management CRUD                  | ✓ Done        | With dept & contract linking         |
| QR Code Generation per Device           | ✓ Done        | Scannable device identification      |
| 3-Tier Maintenance Workflow             | ✓ Done        | Request → WorkOrder → Record         |
| Profile Picture Upload (Cloudinary)     | ✓ Done        | Cloudinary CDN integration           |
| Socket.io Real-time Chat                | ✓ Done        | Configured, UI pending               |
| Automated Cron Reports (node-schedule)  | ✓ Done        | Daily maintenance reminders          |
| Frontend Auth Pages                     | ✓ Done        | Login, Register, ForgotPwd, ResetPwd |
| Devices Page with Filters & Modals      | ✓ Done        | Stats, tabs, search, CRUD modals     |
| User Profile Page                       | ✓ Done        | View info + upload avatar            |
| Postman Test Collection (40+ cases)     | ✓ Done        | All roles & scenarios covered        |
| Maintenance & Work Orders Frontend      | ⌚ In Progress | Planned for final defense            |



| Feature / Module             | Status                                                                                           | Notes                            |
|------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------|
| Dashboard Analytics (Charts) |  <b>Planned</b> | Chart.js / Recharts              |
| AI Chatbot Integration       |  <b>Planned</b> | Device troubleshooting assistant |
| Cloud Deployment             |  <b>Planned</b> | Railway / Vercel / AWS           |

## 7. Future Work

The following features are planned for completion prior to the final defense:

### 7.1 Frontend Completion

- Maintenance Requests page — form submission interface for Staff
- Work Orders page — review and assignment interface for Biomedical Engineers
- Maintenance Records page — execution logging interface for Technicians
- Departments and Contracts management pages
- Dashboard with analytics charts (device status distribution, maintenance trends)

### 7.2 Advanced Features

- AI Chatbot Integration — an assistant trained on biomedical engineering knowledge bases to help staff troubleshoot device issues, understand operating procedures, and request emergency guidance without waiting for an engineer
- QR Code Scanner — mobile-optimized scanning interface for instant device identification on the ward floor
- PDF Report Generation — exportable maintenance history and device inventory reports in PDF and Excel formats
- Push Notifications — browser push notifications for maintenance reminders and fault escalations
- User Roles Management Page — Admin interface for managing user accounts and role assignments

### 7.3 Deployment & DevOps

- Docker containerization (Dockerfile already configured)
- Cloud deployment on Railway (backend) and Vercel (frontend) or AWS
- CI/CD pipeline configuration
- Environment-specific configuration management
- MongoDB Atlas production cluster setup

## 8. References

- [1] Express.js Documentation — <https://expressjs.com/en/5x/api.html>
- [2] MongoDB Mongoose ODM Documentation — <https://mongoosejs.com/docs/>
- [3] React.js Official Documentation — <https://react.dev/>

- [4] JSON Web Tokens (JWT) — RFC 7519 — <https://tools.ietf.org/html/rfc7519>
- [5] Cloudinary Developer Documentation — <https://cloudinary.com/documentation>
- [6] Socket.io Documentation — <https://socket.io/docs/v4/>
- [7] Joi Validation Library — <https://joi.dev/api/>
- [8] NFPA 99: Health Care Facilities Code (Medical Device Maintenance Standards)
- [9] WHO — Medical Device Maintenance: A Guide for Health-care Administrators
- [10] Tailwind CSS Documentation — <https://tailwindcss.com/docs>